ECE 685 Final Projects

Project 1: Hypernetworks for Implicit Neural Representations

Required: 3 people

In this project, we study *implicit neural representations* (INRs) and explore how they can be generated using *hypernetworks*. Unlike conventional neural networks that operate on fixed discrete grids (such as pixels in an image), implicit neural representations model signals as *continuous functions*. This allows signals such as images, audio, or physical fields to be represented as coordinate-based neural networks.

The goal of this project is to investigate how neural networks can learn to generate such representations automatically. In particular, students will implement a *hypernetwork* that takes an image as input and produces the weights of a coordinate network that implicitly represents the image. Students will train and evaluate such models on CIFAR-10 and Celeb-A, and explore whether the learned representations can be useful for downstream tasks.

Implicit Neural Representations

An implicit neural representation models a signal as a function

$$f_\theta : \mathbb{R}^d \to \mathbb{R}^k,$$

where the input corresponds to a spatial coordinate and the output corresponds to the signal value at that coordinate. For example, an image can be represented by a neural network

$$f_\theta(x, y) \to (R, G, B),$$

which maps pixel coordinates (x, y) to color values. Instead of storing image pixels directly, the entire image is encoded in the parameters θ of the neural network. Given an image I, the coordinate network can be trained by minimizing

$$\mathcal{L}(\theta) = \mathbb{E}_{(x,y)} \| f_\theta(x, y) - I(x, y) \|^2,$$

where (x, y) are sampled pixel coordinates.

Implicit neural representations have several attractive properties:

- They represent signals as **continuous functions**.
- They allow evaluation at **arbitrary resolution**.
- They enable differentiation with respect to spatial coordinates.

Classical Architectures for INRs

Several neural architectures have been proposed for implicit representations.

ReLU Coordinate Networks. One simple approach uses a multilayer perceptron (MLP) with ReLU activation that takes spatial coordinates as input. Although easy to implement, such networks often exhibit a *spectral bias*, meaning they tend to learn low-frequency signals before high-frequency ones.

Fourier Feature Networks [Tancik et al., 2020]. To mitigate spectral bias, Fourier feature mappings are often applied to the input coordinates. A typical mapping is

$$\gamma(x) = [\sin(Bx), \cos(Bx)],$$

where B is a frequency matrix. This allows the network to more effectively represent high-frequency signals.

SIREN Networks [Sitzmann et al., 2020]. SIREN (Sinusoidal Representation Networks) replace standard activations with sinusoidal functions

$$\phi(x) = \sin(\omega x).$$

Such networks are particularly effective for representing signals with rich frequency content and have been widely used in implicit representation tasks.

Meta-Learning Implicit Neural Representations

Traditional INR methods require optimizing a new network for each individual signal (e.g., each image). This can be computationally expensive and does not allow the model to generalize across signals.

To address this, we consider a *meta-learning* approach using a hypernetwork. A hypernetwork is a neural network that generates the parameters of another neural network. In our setting, we train a hypernetwork

$$H_\phi(I) \to \theta,$$

which takes an image I as input and outputs the weights θ of a coordinate network f_θ. The full model therefore consists of two components:

- A hypernetwork H_ϕ that maps an image to network parameters.
- A coordinate network f_θ that represents the image as a continuous function.

During training, the hypernetwork is optimized so that the generated coordinate network reconstructs the input image. The objective is

$$\mathcal{L}(\phi) = \mathbb{E}_I \mathbb{E}_{(x,y)} \| f_{H_\phi(I)}(x, y) - I(x, y) \|^2.$$

This framework allows a single model to generate implicit neural representations for an entire dataset.

Task I: CIFAR-10 Meta-Learning and Downstream Tasks

In the first part of the project, students will train a hypernetwork on the CIFAR-10 dataset. The hypernetwork should take an image I as input and generate the parameters of a coordinate network that represents the image.

For this task, students may use a SIREN-based coordinate network as the implicit representation model. After training the hypernetwork, students should explore whether the learned representations are useful for downstream tasks. For example, the generated network weights or latent features from the hypernetwork can be used as inputs to a classifier for CIFAR-10 image classification. Students should compare different feature representations, such as raw pixel features, hypernetwork latent representations, and generated INR weights. The goal is to investigate whether implicit neural representations capture meaningful semantic information.

Task II: Celeb-A Meta-Learning

In the second part of the project, students will apply the same framework to the Celeb-A dataset, which contains higher-resolution face images.

Students should train a hypernetwork that generates implicit neural representations for Celeb-A images. Compared with CIFAR-10, this task will require handling larger images and potentially more complex representations. Students may experiment with different INR architectures, including SIREN networks, Fourier feature networks, ReLU coordinate networks, and *custom architectures designed by the team.*

Students should compare reconstruction quality, training stability, and computational efficiency across different architectures.

Evaluation Metrics. In addition to the standard Peak Signal-to-Noise Ratio (PSNR), students must evaluate their models using additional perceptual quality metrics, including:

- Structural Similarity Index Measure (SSIM)
- Learned Perceptual Image Patch Similarity (LPIPS)

These metrics capture perceptual similarity between images and are commonly used in modern image generation and reconstruction tasks.

Super-Resolution Testing. Students should also evaluate the super-resolution capability of their implicit neural representations. One possible protocol is:

- Downsample Celeb-A images to a lower resolution (e.g., 32×32 or 64×64).
- Train the hypernetwork to generate INRs using the low-resolution supervision.
- Evaluate the reconstructed images at higher resolutions (e.g., 128×128 or 256×256) by querying the coordinate network on a denser grid.

Students should compare the reconstructed high-resolution images against the original images using PSNR, SSIM, and LPIPS.

Deliverables

Each team should submit the following:

- A complete implementation of the proposed models.
- A project report (6–8 pages) describing the methodology, experiments, and findings.
- Experimental results including visual reconstructions and quantitative metrics.

The report should clearly discuss the architecture choices, experimental setup, and insights obtained from the experiments. The report should also analyze how well implicit neural representations generalize to higher resolutions and whether different INR architectures influence super-resolution performance.

Project 2: Denoising Autoencoders for 1D Time-Series Signals

Required: 2 people

In this project, students will instead work with **1D time-series signals**, such as audio, sensor measurements, or biomedical signals. The goal is to design neural networks that can **remove noise from corrupted signals**. The project centers on the concept of a **denoising autoencoder**. Given a clean signal x, we generate a corrupted observation x_{noisy} by injecting noise. A neural network f_θ is then trained to reconstruct the original signal:

$$\hat{x} = f_\theta(x_{\text{noisy}}).$$

The model parameters θ are learned by minimizing a reconstruction loss such as

$$\mathcal{L}(\theta) = \mathbb{E}\big[\|x - f_\theta(x_{\text{noisy}})\|_2^2\big].$$

Beyond basic denoising, this project also explores how different neural architectures model temporal structure.

Datasets. Teams may choose one of the following types of data:

- **Synthetic signals** generated from combinations of sine waves, chirps, or periodic patterns
- **Speech signals** (e.g., Google Speech Commands dataset)
- **Sensor signals** (e.g., accelerometer or wearable device data)
- **Biomedical signals** (e.g., ECG time-series data)

In real datasets, signals should be segmented into fixed-length windows suitable for neural network input.

Noise Models. To create input-target training pairs, students will corrupt clean signals using artificial noise. Each team should implement at least two of the following noise models:

- **Gaussian noise**

$$x_{\text{noisy}} = x + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

- **Random masking (dropout noise)**: randomly remove short segments of the signal.
- **Impulse noise**: randomly insert spikes or outliers into the signal.
- **Sinusoidal interference**: add periodic noise to simulate background interference.

These noise processes simulate common distortions encountered in real-world signals.

Model Architectures

Students will implement and compare at least **two different denoising autoencoder architectures**. Possible choices include:

- **Fully connected autoencoder**
- **Recurrent autoencoder** using LSTM or GRU layers to capture temporal dependencies
- **1D convolutional autoencoder** using temporal convolutions to extract local patterns
- **(Optional) Transformer-based encoder-decoder**

Each architecture follows an encoder-decoder structure:

$$z = \text{Encoder}(x_{\text{noisy}}), \qquad \hat{x} = \text{Decoder}(z),$$

where z is a latent representation with a lower dimension than the input signal.

Experiments and Analysis

Students should conduct systematic experiments to understand how architectural choices affect performance. Suggested experiments include:

- **Comparing different model architectures**: Implement at least two architectures (e.g., MLP, LSTM, or 1D CNN autoencoders) and compare their reconstruction performance and training behavior.
- **Studying the effect of the latent dimension**: Evaluate how different bottleneck sizes influence reconstruction quality and compression ability.
- **Evaluating robustness to different noise types**: Train or test the models under multiple noise models (e.g., Gaussian noise, masking noise, or impulse noise) and compare performance.
- **Testing generalization across noise levels**: Train the model with a fixed noise level and evaluate its denoising performance under different noise intensities.

Evaluation Metrics. Models should be evaluated using quantitative metrics such as:

- **Mean Squared Error (MSE)** between clean and reconstructed signals
- **Signal-to-Noise Ratio (SNR) improvement**

The signal-to-noise ratio can be computed as

$$\mathrm{SNR}(x, \hat{x}) = 10 \log_{10} \left(\frac{\|x\|_2^2}{\|x - \hat{x}\|_2^2} \right).$$

Students should also provide visualizations of reconstructed signals, showing examples of (i) the original clean signal, (ii) the corrupted noisy signal, and (iii) the reconstructed signal produced by the model.

Deliverables

Each team should submit:

- A complete implementation of the models and training pipeline.
- A report describing the dataset, model architectures, and experimental results.
- Quantitative comparisons between the different approaches.
- Visualizations demonstrating denoising performance.

Project 3: Sparse Neural Network Training and Optimization

Required: 2-3 people

Modern neural networks are often highly over-parameterized, containing millions of weights. However, many of these parameters are redundant, and recent research has shown that neural networks can often be trained effectively even when most connections are removed. Instead of first training a dense network and then pruning it, a newer paradigm is to **train sparse neural networks directly from scratch**. In such models, only a small subset of weights are active during training, which can greatly reduce memory usage and computation.

In this project, students will study **sparse neural network training** and investigate how different optimization strategies behave under extreme sparsity. Let W denote the weight matrix of a neural network. Sparse training introduces a binary mask M such that

$$W = M \odot \theta,$$

where $\odot$ denotes elementwise multiplication and θ are the underlying trainable parameters. Entries where $M_{ij} = 0$ correspond to inactive connections that remain zero throughout training.

Students will train sparse convolutional neural networks on the CIFAR-10 dataset and explore three increasingly sophisticated approaches: static sparse training, Sparse Evolutionary Training (SET), and RigL.

Dataset and Model

The experiments will be conducted on the **CIFAR-10** image classification dataset. Students should train a small convolutional neural network such as **ResNet-20** (or a comparable lightweight ResNet architecture). To study the effect of sparsity, models should be trained at several sparsity levels, for example:

- **90% sparsity** (10% active weights)
- **95% sparsity**
- **98% sparsity**

The goal is to understand how network performance changes as the number of active parameters decreases.

Task 1: Sparse Networks from Scratch

In the first task, students will train sparse networks with a **fixed random connectivity mask**. The mask M is generated at initialization and remains unchanged throughout training. Students should compare at least two optimization methods: **SGD with momentum** and **Adam**. The goal is to investigate how different optimizers behave when training highly sparse networks. Suggested questions include:

- How does sparsity affect convergence speed?
- How does accuracy degrade as sparsity increases?
- Do different optimizers behave differently under high sparsity?

Task 2: Sparse Evolutionary Training (SET)

In the second task, students will implement **Sparse Evolutionary Training (SET)** [Mocanu et al., 2018], which allows the sparse connectivity pattern to evolve during training. At regular intervals during training:

- Remove a fraction of the smallest magnitude weights.
- Add the same number of new connections at random locations.

This procedure maintains a constant sparsity level while enabling the network structure to adapt over time. Students should compare the performance of SET with the static sparse baseline.

Task 3: Rigging and Lottery (RigL)

In the third task, students will implement a simplified version of **Rigging the Lottery (RigL)** [Evci et al., 2020], which improves dynamic sparse training by using gradient information to guide the growth of new connections. During topology updates:

- Remove weights with the smallest magnitude.
- Add new connections at locations with the largest gradient magnitude.

The intuition is that connections with large gradients would most effectively reduce the loss if they were active. Students should compare RigL with both static sparse training and SET.

Experiments and Analysis

Students should conduct systematic experiments to understand how sparsity affects optimization and model performance. Suggested experiments include:

- **Accuracy vs. sparsity**: Evaluate model performance at multiple sparsity levels.
- **Optimizer comparison**: Compare the training behavior of SGD and Adam under sparse training.
- **Static vs. dynamic sparsity**: Compare fixed sparse training, SET, and RigL.
- **Visualization of convolution kernels**: Visualize learned convolution filters to examine how sparsity is distributed across kernels and layers.
- **Connectivity analysis**: Analyze how sparse connectivity evolves during training in SET or RigL.

Students are encouraged to produce clear plots such as training curves, accuracy vs. sparsity graphs, and visualizations of convolution filters.

Evaluation Metrics. Models should be evaluated using standard classification metrics such as:

- **Test accuracy** on CIFAR-10
- **Training loss and validation accuracy curves**

In addition, students should quantify the **sparsity level** of the trained networks. Let W denote a weight tensor and let $\|W\|_0$ denote the number of nonzero entries. The sparsity level of a layer can be defined as

$$\text{sparsity}(W) = 1 - \frac{\|W\|_0}{|W|},$$

where $|W|$ is the total number of parameters in the layer. Students should report sparsity statistics for different components of the network, for example:

- **Connection sparsity**: the proportion of active weights in each layer.
- **Convolution kernel sparsity**: the proportion of nonzero parameters within convolutional filters.
- **Fully connected layer sparsity**: the proportion of active connections in fully connected layers.

For the methods with dynamic connectivity (SET and RigL), students are encouraged to **track sparsity statistics during training**.

Deliverables

Each team should submit:

- A complete implementation of the sparse training algorithms.
- A report describing the models, methods, and experimental results.
- Quantitative comparisons between static sparse training, SET, and RigL.
- Visualizations illustrating sparsity patterns and learned convolution kernels.

Project 4: Video Inpainting with Unconditional Priors and Conditional Guidance

Required: 3 people

Video inpainting aims to reconstruct missing regions in a video sequence in a way that is both visually plausible and temporally consistent. Compared with image inpainting, video inpainting is more challenging because the model must maintain stable appearance and motion across time.

In this project, students will study **video inpainting using a pretrained unconditional generative model**. Instead of training a generative model from scratch, students will use a pretrained model as a **prior** over realistic videos and focus on designing effective **inference methods** for reconstructing missing regions.

Problem Formulation

Let $V = \{v_t\}_{t=1}^T$ be a clean video and let $M = \{m_t\}_{t=1}^T$ be a binary mask sequence. The observed corrupted video is

$$\tilde{v}_t = (1 - m_t) \odot v_t + m_t \odot c_t.$$

The goal is to infer missing pixels Y given observed pixels X, i.e.,

$$p(Y \mid X).$$

Students will approximate this conditional distribution using a pretrained model $p_\theta(V)$:

$$p(Y \mid X) \propto p_\theta(X, Y).$$

Task I: Generative Model for Video Prior

Students will use a generative model $p_\theta(V)$ to represent a prior over realistic videos.

Pretrained model. Students are encouraged to use an existing pretrained generative model (e.g., GAN, VAE, or autoencoder-based model) as a prior. This allows the project to focus on **inference and conditioning**, rather than model training.

Students should clearly describe:

- what model they use,
- whether it is pretrained or trained from scratch,
- what distribution it is intended to represent.

Task II: Conditional Inpainting Baseline (No Guidance)

Students will implement a conditional generative model that reconstructs masked frames without using the unconditional prior. Two options include **Conditional VAE** and **Conditional GAN**. An example reconstruction loss is

$$\mathcal{L}_{\text{rec}} = \frac{1}{T} \sum_{t=1}^{T} \left\| m_t \odot (\hat{v}_t - v_t) \right\|_1 .$$

Students are also encouraged include a lightweight temporal component (e.g., ConvLSTM or temporal attention), but this is not the main focus of the task.

Task III: Guidance by the Unconditional Prior

Students will design a mechanism that uses the unconditional model from Task I to guide the conditional model from Task II. A recommended starting point is to incorporate the unconditional model as a **prior**

regularization term in the training objective. For example, students may consider an objective of the form

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda\, \mathcal{L}_{\text{prior}},$$

where $\mathcal{L}_{\text{prior}}$ encourages reconstructed videos to lie in the distribution learned by the unconditional model. We list some design choices below, while other designs are also encouraged:

- **Latent prior guidance:** encourage conditional latent variables to match those from the unconditional model (e.g., via KL or ℓ_2 penalties).
- **Discriminator approach:** Train a discriminator on complete video clips to distinguish real videos from reconstructed ones. The discriminator acts as an unconditional prior enforcing realism:

  $$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{dis}}.$$

  Students may use either frame-wise or clip-level discriminators, though clip-level designs are encouraged for better temporal consistency (other loss functions are also encouraged).
- **Energy-based guidance:** treat the unconditional model as defining a prior energy and penalize unrealistic outputs.

Students must implement at least one guidance mechanism. More advanced designs are encouraged but not required. Students should introduce a **guidance strength parameter** λ and study its effect.

Minimum requirement. At least one working implementation of prior-guided training must be demonstrated, even if the design is simple. A *key question* here is: How does increasing λ affect:

- reconstruction accuracy (e.g., SSIM [Wang et al., 2004]),
- perceptual realism (e.g., LPIPS [Zhang et al., 2018]),
- temporal consistency (e.g., TecoGAN-style temporal coherence metrics [Chu et al., 2020]),
- diversity of outputs (e.g., DRIT-style diversity evaluation [Lee et al., 2018])?

Task IV: Analysis of Spatial and Temporal Quality under Guidance

Students will conduct a systematic evaluation of how prior-guided training affects the quality of reconstructed videos. In particular, they should compare (i) the conditional baseline without guidance, and (ii) the guided model under multiple values of the guidance strength parameter λ. Optionally, students may also include a temporally enhanced variant for additional comparison. Evaluation should consider both **spatial quality** and **temporal consistency**:

- For spatial quality, students should report standard image-based metrics like PSNR, SSIM, and LPIPS.
- For temporal consistency, they should include at least one metric that captures frame-to-frame coherence, such as warping error, frame-difference consistency, or a perceptual temporal metric.

Beyond reporting metrics, students must analyze how performance changes as the guidance strength λ varies. In particular, they should investigate whether guidance improves perceptual realism, whether overly strong guidance leads to over-smoothing or loss of fine details, and whether temporal stability improves as a consequence of better global structure.

The report must include both quantitative results (e.g., tables or plots as a function of λ) and qualitative visualizations that highlight typical successes and failure cases.

Project 5: Image Inpainting (3-4 students)

Problem Formulation

Let $x \sim p_X$ denote a natural image. A rectangular corruption mask M is a random variable is applied to generate a set of corrupted images.

The rectangular mask $M \in \{0,1\}^{H \times W}$ is a random variable parameterized by center (c_x, c_y) and dimensions (h, w), where

$$h \sim \text{Uniform}([h_{\min}, h_{\max}]), \quad w \sim \text{Uniform}([w_{\min}, w_{\max}])$$
$$c_x \sim \text{Uniform}\left(\frac{w}{2},\ W - \frac{w}{2}\right), \quad c_y \sim \text{Uniform}\left(\frac{h}{2},\ H - \frac{h}{2}\right)$$

where h and w are drawn first to ensure the center is chosen such that the mask lies entirely within the image boundary. The mask takes values

$$M_{ij} = \begin{cases} 1 & \text{if } |i - c_y| \leq \frac{h}{2} \ \text{ and } \ |j - c_x| \leq \frac{w}{2} \\ 0 & \text{otherwise} \end{cases}$$

so that masked pixels are entirely contained within $[0, H] \times [0, W]$. The corrupted image is then

$$y = x \odot (1 - M)$$

where $\odot$ denotes element-wise multiplication, the masked pixels ($M_{ij} = 1$) are zeroed out, and the unmasked pixels ($M_{ij} = 0$) are observed exactly.

Image inpainting is the problem of learning the conditional distribution $p_{\hat{X}|Y}$ (or $p_{\hat{X}|Y,M}$). The distortion of the algorithm is $\mathbb{E}[\Delta(X, \hat{X})]$ for some dissimilarity Δ, and its perceptual quality is $d(p_X, p_{\hat{X}})$ for some distributional divergence d. Blau & Michaeli (CVPR 2018) prove that these two quantities are fundamentally at odds: no algorithm can simultaneously minimize distortion and achieve $p_{\hat{X}} = p_X$. This tradeoff motivates the evaluation framework below.

Dataset

CelebA-HQ — 30,000 face images at 1024×1024. Used for training and primary evaluation.

FFHQ (bonus) — 70,000 face images at 1024×1024. Used for generalization testing only — no training on FFHQ.

Evaluation Setup

Evaluation masks are rectangular, with center (c_x, c_y) drawn uniformly over the image and dimensions (h, w) drawn uniformly from three size regimes:

Condition	**Height/width range**	**Approx. coverage**
Small	$[0.1H, 0.2H] \times [0.1W, 0.2W]$	1–4%
Medium	$[0.3H, 0.4H] \times [0.3W, 0.4W]$	9–16%
Large	$[0.5H, 0.6H] \times [0.5W, 0.6W]$	25–36%

Methods

Implement three methods: one must be reconstruction-based and one must be GAN-based. The third is your choice from the generative family.

Reconstruction-based. Treats inpainting as pixel regression: $\hat{x} = f_\theta(y)$ or $f_\theta(y, M)$, trained to minimise a pixel-level or feature-level loss against the ground truth x. The output is deterministic.

References: Pathak et al. *Context Encoders* (CVPR 2016); Liu et al. *Partial Convolutions* (ECCV 2018).

Generative. Treats inpainting as conditional image generation. Learns a distribution over plausible completions (output is stochastic).

GANs

The generator G defines $p_{\hat{X}|Y}$ and is trained adversarially so that $p_{\hat{X}} \approx p_X$. The output is stochastic.

References: Yeh et al. *Semantic Image Inpainting* (CVPR 2017); Yu et al. *DeepFill* (CVPR 2018).

Non-GAN based (your choice). Any conditional generative model that learns a distribution over completions without adversarial training.

References: Zheng et al. *Pluralistic Image Completion* (CVPR 2019); Kingma & Welling *Auto-Encoding Variational Bayes* (ICLR 2014).

Training Paradigm

All three methods require a choice of conditioning strategy. You may condition on (y, M) together, on y alone, or (for generative methods) train an unconditional model on clean images and transfer to inpainting at inference. Your choice of masks while training can also differ from the evaluation setup (training masks can be a superset of evaluation masks). Each choice has implications for what the model knows about the missing region, how it generalizes to unseen mask distributions, and computational cost. Justify your choice for each method.

Metrics

Report MSE, SSIM, and FID for all three methods across all mask conditions.

Classify each of MSE, SSIM, and FID: is it a distortion metric (measures $\mathbb{E}[\Delta(X, \hat{X})]$) or a perception metric (measures $d(p_X, p_{\hat{X}})$) in the sense of Blau & Michaeli?

Identify (at least one) additional perception-metric that you can implement for this project. Justify your choice.

Analysis: The Perception-Distortion Tradeoff

Using your four metrics, place each of your three methods on the perception-distortion plane. Then analyze:

- As mask size increases, how do distortion and perception metrics change for each method? Do they move in the same direction or opposite directions?
- Which of your methods lies closest to the perception-distortion bound, and what does that imply about its design?

Bonus: Generalisation to FFHQ

Apply all three trained models zero-shot to FFHQ using the medium mask condition. Report all four metrics. Does the perception-distortion relationship hold under distribution shift? Which paradigm generalises better in terms of distortion, and which in terms of perception and why?

Project 6: Efficient NN Classifiers via Sparse Mixture of Experts (3-4 students)

Every year, millions of camera traps quietly photograph the world's wildlife : jaguars crossing forest floors, rare orchids flowering for a single week, invasive species spreading undetected. The images pile up faster than ecologists can analyze them. Automating species identification could transform biodiversity conservation, but only if the model can run *on the camera itself*. A server-side classifier is useless when the trap is in a rainforest with no connectivity. This demands models that are not just accurate, but inference-efficient: capable of running under tight memory and compute budgets without sacrificing the ability to distinguish visually similar species.

Standard dense networks apply the same parameters to every input : the same weights fire whether the image shows a hummingbird or a whale. This is potentially wasteful. A sparse Mixture of Experts (MoE) model can exploit this structure by routing each image to a small subset of specialized expert networks, keeping inference cost low while maintaining a large total parameter capacity.

Dataset

iNaturalist 2019 — 268,243 images across 1,010 species spanning plants, animals, fungi, and more. Heavily class-imbalanced, fine-grained, and visually challenging. Available via Kaggle (FGVC6 competition). The dataset's taxonomic hierarchy (kingdom → phylum → class → order → family → genus → species) provides natural sub-structure for expert specialization.

Baseline

EfficientNet-B0 fine-tuned on iNaturalist 2019. EfficientNet uses compound scaling to jointly optimize network depth, width, and input resolution — making it the canonical lightweight architecture for resource-constrained classification. Familiarize yourself with this architecture before designing your MoE model.

Reference: Tan & Le, *EfficientNet* (ICML 2019).

Method

Starting from EfficientNet-B0, design and implement a sparse MoE variant. You must make and justify the following design decisions:

- **Where to insert MoE layers.** Which layers or blocks do you replace with MoE layers, and why? Consider what different layers encode (low-level texture vs. high-level semantics) and where routing is likely to be meaningful.
- **Number and size of experts.** More experts increase capacity but add parameters. Smaller experts reduce cost but may underfit. Justify your choice relative to the deployment constraint.
- **Routing mechanism.** How does the gating network decide which expert handles each input? What routing strategy (top-k, soft, learned) did you choose and why? How do you prevent expert collapse?
- **Training objective.** Do you use an auxiliary load-balancing loss? How do you balance routing stability against classification accuracy?

References: Shazeer et al. *Sparsely-Gated MoE* (ICLR 2017); Fedus et al. *Switch Transformers* (JMLR 2022); Riquelme et al. *V-MoE* (NeurIPS 2021).

Evaluation

Report top-1 accuracy, total parameter count, and activated parameters per inference (FLOPs). Your model must not exceed the total parameter count of EfficientNet-B0 by more than 20%, while activating *fewer* parameters per sample at inference.

Your report must argue: do different experts specialize by taxonomic group? Visualize routing patterns across the iNaturalist super-categories (birds, mammals, plants, etc.): does the gating network learn taxonomy?

Project 7: Pediatric CXR Classification via Domain Adaptation (2 people)

Current radiology AI models are trained almost exclusively on adult chest X-rays, leaving pediatric diagnosis severely underserved. Pediatric CXRs differ from adult CXRs in anatomy, disease presentation, and imaging protocols, yet labeled pediatric data is scarce. This project frames the problem as domain adaptation: leverage abundant labeled adult CXRs to improve classification on a small labeled pediatric dataset.

Datasets

- **NIH ChestX-ray14** (adult, source domain) — 112,120 frontal-view X-rays, simplified to binary: pneumonia vs. normal. Wang et al., CVPR 2017.
- **Kermany et al. Pediatric CXR** (target domain) — 5,863 images, binary labels (Pneumonia / Normal), children aged 1–5. Available on Kaggle.

Method

Stage 1 — Contrastive pretraining on adult data. Train a dual-encoder model aligning adult CXR images with class-level text prompts (e.g. *"a chest X-ray showing pneumonia"*) using a contrastive loss (e.g. InfoNCE). The image encoder learns a latent space structured around pathology semantics.

Stage 2 — Pediatric fine-tuning. Freeze the text encoder. Fine-tune the image encoder on the small labeled pediatric set, anchoring pediatric CXR representations to the adult-learned text embeddings via a contrastive alignment loss.

Stage 3 — Classification. Classify by nearest text prompt embedding in latent space.

References: Radford et al. *CLIP* (ICML 2021); Tiu et al. *CheXzero* (Nature Biomedical Engineering 2022); Chen et al. *SimCLR* (ICML 2020).

The baseline is a classifier trained on pediatric data only, with no adult pretraining. Beyond the proposed method, read the literature and propose at least one alternative strategy for aligning adult and pediatric representations in latent space. Justify your choice.

Evaluation

Report AUC-ROC and F1 on the pediatric test set. Vary the size of the pediatric fine-tuning set and report the performance curve. Visualize the latent space (t-SNE) before and after fine-tuning to verify that alignment occurs. Your report must argue: does adult contrastive pretraining help, and under what conditions does the gap with the baseline close?

Project 8: Learning Under Symmetry Constraints

In physics and chemistry, many problems are invariant to the change of coordinates and position. For instance, the enthalpy of a reaction does not depend upon the reaction's location, and any two objects attract each other with a gravitational force which is invariant to the changes of coordinates of the system Cohen and Welling [2016], Kondor [2025]. Recall from linear algebra that any linear transformation corresponding to a change of coordinates in $\mathbb{R}^l$ can be realized by multiplication by an orthogonal matrix $U \in \mathbb{U}(m)$, where $\mathbb{U}(l)$ denotes the set of $l \times l$ orthogonal matrices.

Next, let us consider a special case linear factor model (LFM) given by $Y = AX$ where $X \in \mathbb{R}^d$ is the image and $Y \in \mathbb{R}^m$ is the latent representation of X and $A \neq 0$ is an $m \times d$ matrix. We are interested in a representation that is *equivariant* to the change of coordinates in the sense that a change of coordinates in the image domain corresponds to another change of coordinates in the latent domain.

In other words, associated with each $U \in \mathbb{U}(d)$, there exists $V \in \mathbb{U}(m)$ such that

$$VY = AUX. \tag{0.1}$$

for all $X \in \mathbb{R}^d$ and $Y = AX$. This implies that

$$VA = AU$$

for the pair (U, V).

- (i) Provide sufficient conditions on (U, V) such that (0.1) holds for a nonzero matrix A.
- (ii) Is it possible to construct a matrix A such that any change of coordinates in the image domain corresponds to a change of coordinates in the latent domain?
- (iii) Consider orthogonal matrices U_1 and V_1 such that there exists an $n \geq 1$ with $U_1^n = I_d$ and $V_1^n = I_m$ where I_d and I_m are respectively identity matrices in d and m dimensions. Given $n > 1$, construct an example of such U_1 and V_1 such that the condition of (i) holds and $V_1 A = AU_1$ for some $A \neq 0$. Then it is easy to see (show this) that $V_j A = AU_j$ for all $j \geq 1$ where $U_j = U_1^j$ and $V_j = V_1^j$. For your example of U_1 and V_1, give an algorithm (and implement it) to construct the transformation A.

Next, we consider standard MNIST images with x and y coordinates parallel to the sides of the images, and the origin in the center of the images (See Figure 1). We restrict the change of coordinates transformations of interest to correspond to rotations of $0, 90, 180$ and 270 degrees, reflections in x and y axes and their compositions.

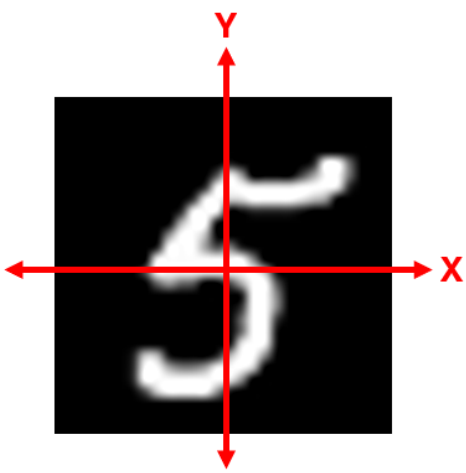
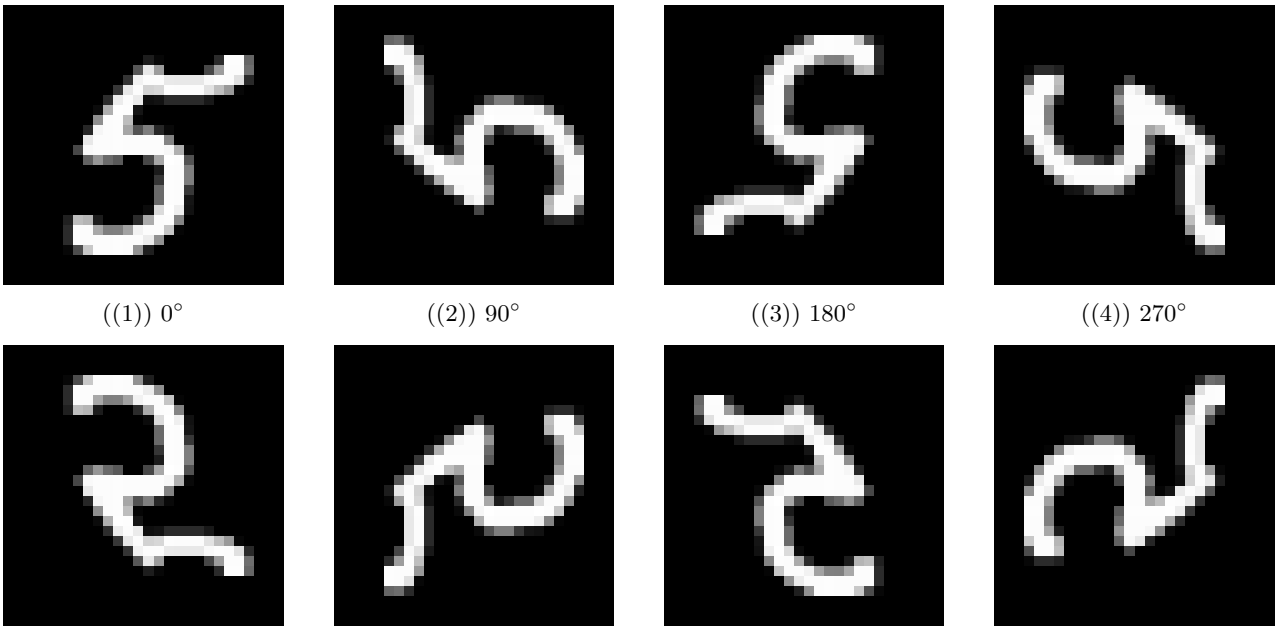


Figure 1: An MNIST image with x- and y-axes.

For any given image, the pixel at position (i, j) will be moved to position $(\tilde{i}, \tilde{j})$ following a rotation or reflection. The map from (i, j) to $(\tilde{i}, \tilde{j})$ under rotation is given by matrices $T_{0^\circ}, T_{90^\circ}, T_{180^\circ}$, and T_{270°, and map under rotation composed with reflection are given by matrices $R_{0^\circ}, R_{90^\circ}, R_{180^\circ}$, and R_{270°:

$$T_{0^\circ} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad T_{90^\circ} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad T_{180^\circ} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}, \quad T_{270^\circ} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix},$$
$$R_{0^\circ} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_{90^\circ} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad R_{180^\circ} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad R_{270^\circ} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}, \tag{0.2}$$



((1)) 0° ((2)) 90° ((3)) 180° ((4)) 270°

((5)) reflect ((6)) 90° + reflect ((7)) 180° + reflect ((8)) 270° + reflect

Figure 2: Rotations and Reflections of MNIST Digits

- Show that for an element B in this set of transformations, $B^4 = I_{784}$.
- Design an equivariant linear generalized autoencoder (with both equivariant linear paired encoder and decoder) for this set of transformations where $d = 784 = 28 \times 28$ (MNIST dimension) for $m = 40$. A generalized autoencoder is a neural network that upon input of any image, outputs a rotated, reflected, or rotated and reflected version of the input image. Also, recall from the course material that in a paired linear autoencoder, the encoder and decoder matrices are transposed of each other. If this is not possible, construct a linear generalized autoencoder that is approximately equivariant for these transformations. Propose your own definition of "approximate equivariance."
- Is it possible to construct a nonlinear generalized autoencoder that is equivariant for the above set of transformations? If yes, write an algorithm (and implement it) to construct one. If not, construct a nonlinear generalized autoencoder that is approximately equivariant to these transformations. Propose your own definition of "approximate equivariance."
- Use the above generalized autoencoder for classification of rotated and reflected MNIST images. Compare the performance of your construction to those of baseline autoencoders for MNIST classification.

References

Mengyu Chu, You Xie, Jonas Mayer, Laura Leal-Taixé, and Nils Thuerey. Learning temporal coherence via self-supervision for gan-based video generation. *ACM Transactions on Graphics (TOG)*, 39(4):75–1, 2020.

Taco Cohen and Max Welling. Group equivariant convolutional networks. In *International conference on machine learning*, pages 2990–2999. PMLR, 2016.

Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. Rigging the lottery: Making all tickets winners. In *International conference on machine learning*, pages 2943–2952. PMLR, 2020.

Risi Kondor. The principles behind equivariant neural networks for physics and chemistry. *Proceedings of the National Academy of Sciences*, 122(41):e2415656122, 2025.

Hsin-Ying Lee, Hung-Yu Tseng, Jia-Bin Huang, Maneesh Singh, and Ming-Hsuan Yang. Diverse image-to-image translation via disentangled representations. In *Proceedings of the European conference on computer vision (ECCV)*, pages 35–51, 2018.

Decebal Constantin Mocanu, Elena Mocanu, Peter Stone, Phuong H Nguyen, Madeleine Gibescu, and Antonio Liotta. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature communications*, 9(1):2383, 2018.

Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in neural information processing systems*, 33: 7462–7473, 2020.

Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems*, 33:7537–7547, 2020.

Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004.

Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018.